

AFRILANG Stdlib

std/c/nlpsimhash

Français. Bibliothèque AFRILANG 'std/c/nlpsimhash' (niveau complexe, catégorie complexe). Elle expose 7 fonction(s) : simhLen, simhUpper, simhLower, simhTrim, simhSplit, simhJoin, simhReplace.

```
'import "std/c/nlpsimhash"' · 'use nlpsimhash'
```

```
- 'simhLen(s text) → number' - 'simhUpper(s text) → text' - 'simhLower(s text) → text' - 'simhTrim(s text) → text' - 'simhSplit(s text, delim text) → list text' - 'simhJoin(parts list text, delim text) → text' - 'simhReplace(s text, from text, to text) → text'
```

English. AFRILANG library 'std/c/nlpsimhash' (tier complex, category complex). Exports 7 function(s): simhLen, simhUpper, simhLower, simhTrim, simhSplit, simhJoin, simhReplace.

```
'import "std/c/nlpsimhash"' · 'use nlpsimhash'
```

```
- 'simhLen(s text) → number' - 'simhUpper(s text) → text' - 'simhLower(s text) → text' - 'simhTrim(s text) → text' - 'simhSplit(s text, delim text) → list text' - 'simhJoin(parts list text, delim text) → text' - 'simhReplace(s text, from text, to text) → text'
```

Catégorie / Category	Modules complex (c/)
Niveau / Tier	complex
Fonctions / Functions	7

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/nlpsimhash' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : simhLen, simhUpper, simhLower, simhTrim, simhSplit, simhJoin, simhReplace.

'import "std/c/nlpsimhash"' · 'use nlpsimhash'

```
- `simhLen(s text) → number`
- `simhUpper(s text) → text`
- `simhLower(s text) → text`
- `simhTrim(s text) → text`
- `simhSplit(s text, delim text) → list text`
- `simhJoin(parts list text, delim text) → text`
- `simhReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/nlpsimhash"
use nlpsimhash
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- simhLen(s text) → number•
simhUpper(s text) → text•
simhLower(s text) → text•
simhTrim(s text) → text•
simhSplit(s text, delim text) → list•
simhJoin(parts list text, delim text) → text•
simhReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/nlpsimhash"
use nlpsimhash
```

```
create result = simhLen(/* args */)
say result
```

```
create result = simhUpper(/* args */)
say result
```

```
create result = simhLower(/* args */)
say result
```

```
create result = simhTrim(/* args */)
say result
```

```
create result = simhSplit(/* args */)
say result
```

```
create result = simhJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/nlpsimhash"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module nlpsimhash

```
function simhLen(s text) returns number
end
```

```
function simhUpper(s text) returns text
end
```

```
function simhLower(s text) returns text
end
```

```
function simhTrim(s text) returns text
end
```

```
function simhSplit(s text, delim text) returns list text
end
```

```
function simhJoin(parts list text, delim text) returns text
end
```

```
function simhReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/nlpsimhash' (tier complex, category complex). Exports 7 function(s): simhLen, simhUpper, simhLower, simhTrim, simhSplit, simhJoin, simhReplace.

```
'import "std/c/nlpsimhash"' · 'use nlpsimhash'
```

```
- `simhLen(s text) → number`
- `simhUpper(s text) → text`
- `simhLower(s text) → text`
- `simhTrim(s text) → text`
- `simhSplit(s text, delim text) → list text`
- `simhJoin(parts list text, delim text) → text`
- `simhReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/nlpsimhash"
use nlpsimhash
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- simhLen(s text) → number•
simhUpper(s text) → text•
simhLower(s text) → text•
simhTrim(s text) → text•
simhSplit(s text, delim text) → list•
simhJoin(parts list text, delim text) → text•
simhReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/nlpsimhash"
use nlpsimhash
```

```
create result = simhLen(/* args */)
say result
```

```
create result = simhUpper(/* args */)
say result
```

```
create result = simhLower(/* args */)
say result
```

```
create result = simhTrim(/* args */)
say result
```

```
create result = simhSplit(/* args */)
say result
```

```
create result = simhJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/nlpsimhash"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module nlpsimhash

```
function simhLen(s text) returns number
end
```

```
function simhUpper(s text) returns text
end
```

```
function simhLower(s text) returns text
end
```

```
function simhTrim(s text) returns text
end
```

```
function simhSplit(s text, delim text) returns list text
end
```

```
function simhJoin(parts list text, delim text) returns text
end
```

```
function simhReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/nlpsimhash"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/nlpsimhash/>

Source (excerpt)

```
module nlpsimhash

function simhLen(s text) returns number
end

function simhUpper(s text) returns text
end

function simhLower(s text) returns text
end

function simhTrim(s text) returns text
end

function simhSplit(s text, delim text) returns list text
end

function simhJoin(parts list text, delim text) returns text
end

function simhReplace(s text, from text, to text) returns text
end
end
```